Name: Soham Chavan

PRN: 124B1F022

Class: IT A

Assignment No. 3

Aim:

To implement the Fractional Knapsack algorithm using a Greedy approach to maximize the total utility value of relief supplies carried by a boat of limited weight capacity, allowing partial selection of divisible items.

Code:

```
import java.util.Arrays;
import java.util.Comparator;

class ReliefItem {
    String name;
    double weight;
    double value;
    double ratio;

    ReliefItem(String name, double weight, double value) {
        this.name = name;
        this.weight = weight;
        this.value = value;
        this.ratio = value / weight;
    }
}

public class FractionalKnapsackRelief {

    public static void fractionalKnapsack(ReliefItem[] items, double capacity) {
        Arrays.sort(items, Comparator.comparingDouble((ReliefItem i) -> i.ratio).reversed());

        double remainingCapacity = capacity;
        double totalValue = 0;

        System.out.println("Item selection (boat capacity = " + capacity + " kg):\n");

        for (ReliefItem item : items) {
            if (remainingCapacity <= 0) break;

            if (item.weight <= remainingCapacity) {
                remainingCapacity -= item.weight;
                totalValue += item.value;
                System.out.println("Taken FULLY: " + item.name + " | Weight:" + item.weight +
" | Value:" + item.value);
            } else {
                double fraction = remainingCapacity / item.weight;
```

```
                totalValue += item.value * fraction;
                System.out.println("Taken PARTIALLY: " + item.name + " | Fraction:" +
String.format("%.2f", fraction)
                        + " | Weight taken:" + String.format("%.2f", remainingCapacity) + " |
Value gained:" + String.format("%.2f", item.value * fraction));
                remainingCapacity = 0;
            }
        }

        System.out.println("\nTotal utility value carried by the boat: " +
String.format("%.2f", totalValue));
    }

    public static void main(String[] args) {
        ReliefItem[] items = {
            new ReliefItem("Medicine Kits", 20, 200),
            new ReliefItem("Drinking Water", 50, 160),
            new ReliefItem("Food Packets", 40, 120),
            new ReliefItem("Blankets", 30, 90),
            new ReliefItem("Tents", 60, 150)
        };

        double boatCapacity = 100;

        fractionalKnapsack(items, boatCapacity);
    }
}
```

Output:

```
Item selection (boat capacity = 100.0 kg):

Taken FULLY: Medicine Kits | Weight:20.0 | Value:200.0
Taken FULLY: Drinking Water | Weight:50.0 | Value:160.0
Taken PARTIALLY: Food Packets | Fraction:0.75 | Weight taken:30.00 | Value gained:90.00

Total utility value carried by the boat: 450.00
```

Conclusion:

The Fractional Knapsack problem was solved using a Greedy strategy by sorting items on their value-to-weight ratio and picking items with the highest ratio first, taking a fraction of an item when the full item does not fit. This greedy approach gives an optimal solution for the fractional knapsack problem with a time complexity of O(n log n), dominated by the sorting step, and ensures maximum utility value is transported within the boat's weight limit W.